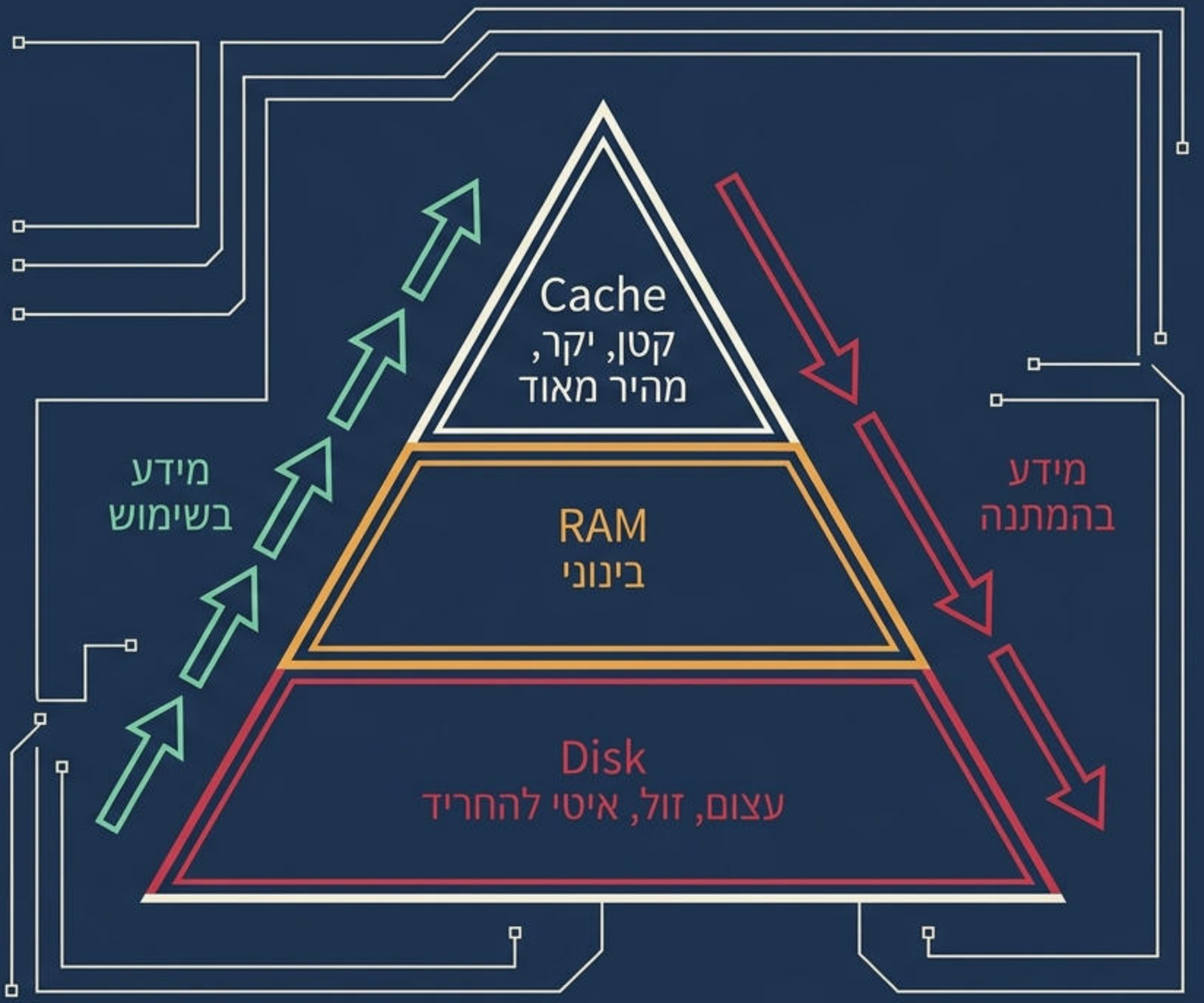
ארכיטקטורת הזיכרון: פרדוקס המהירות והנפח

הבעיה הפיזיקלית:
המעבד דורש נתונים באופן מיידי, אך זיכרון מהיר עולה הון ומוגבל בנפחו.

הפתרון המערכתי:
היררכיית זיכרון מרובת-רמות.
מערכת ההפעלה משנעת מידע בזמן אמת אל שכבות הזיכרון העליונות כדי לייצר אשליה של זיכרון עצום ומהיר.

כלל אצבע:
מידע בשימוש פעיל עובר מעלה;
מידע בהמתנה שוקע מטה.



מדידת ביצועים: זמן גישה משוקלל (EMAT)

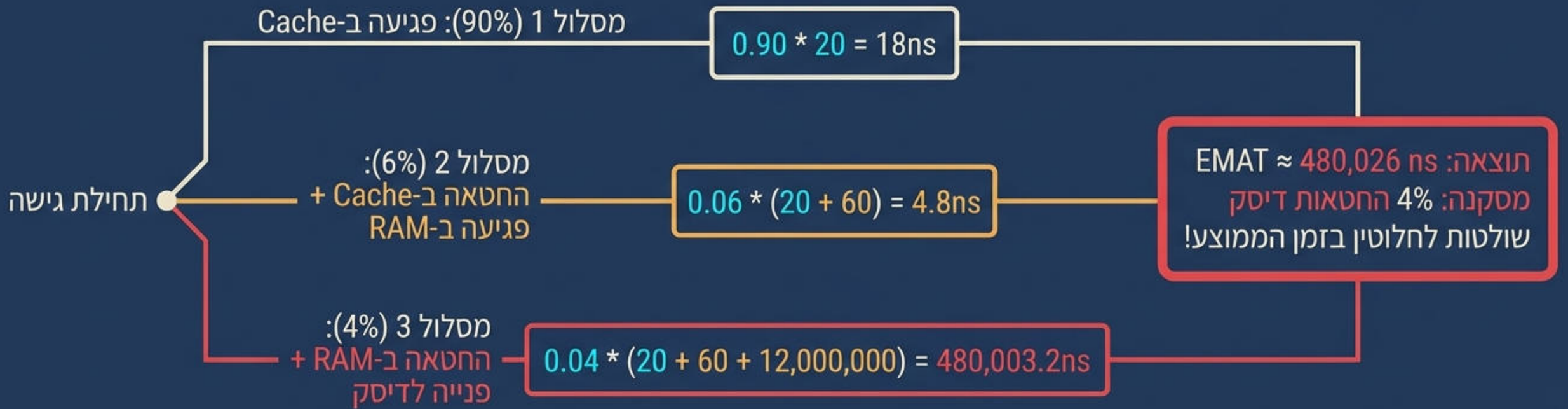
$$EMAT = H \cdot T_{fast} + (1 - H) \cdot (T_{fast} + T_{slow})$$

H = יחס ההצלחה (Hit Ratio)

מעבדת מתמטיקה: חישוב ב-3 רמות

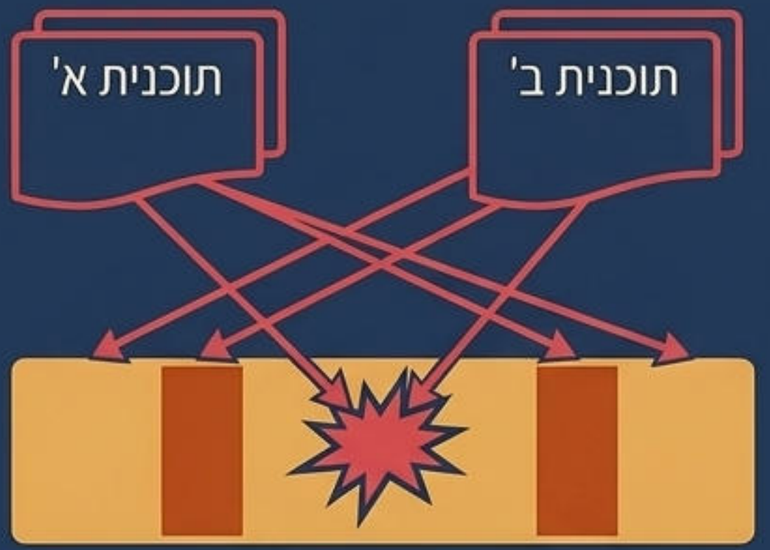
נתונים:

Cache = 20ns (Cyan), RAM = 60ns (Amber), Disk = 12,000,000ns (Red)



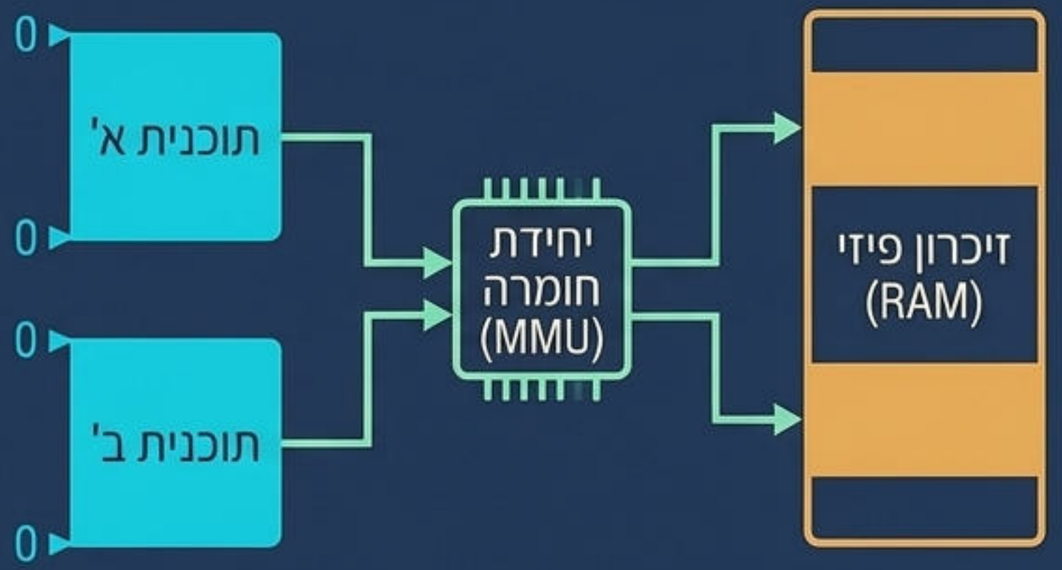
הולדת ההפשטה: מניהול ישיר למרחב כתובות מאובטח

ניהול זיכרון ישיר (ללא הפשטה)



תוכניות מנסות לגשת לאותה כתובת פיזית במקביל
במקביל ודורסות זו את זו.
אין יכולת למולטיפרוגרמינג מאובטח.

מרחב כתובות (Address Space)



כל תהליך מקבל אשליה של זיכרון פרטי רציף.
הגישה מפוענחת בזמן אמת.

Physical Address = Logical Address + Base Register

Limit Register: מגדיר את אורך התהליך למניעת חריגות

מצביע על התחלת הכתובת ב-RAM

החלפה לדיסק (Swapping) וניהול זיכרון פנוי

הבעיה: פרגמנטציה חיצונית - שברים של זיכרון פנוי שאינם רציפים, המונעים טעינת תוכנית גדולה.



רשימות מקושרות (Linked Lists)



ציון כתובת התחלה ואורך מקטע.

יתרון: מהיר משמעותית לסריקה.

מפת סיביות (Bitmaps)



0 = פנוי, 1 = תפוס.

חיסרון: סריקה אטית של רצפי אפסים למציאת מקום.

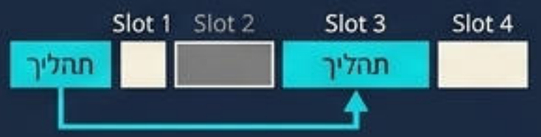
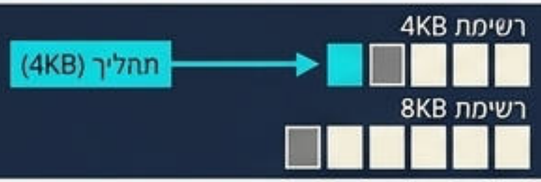
חוק ה-50% של קנות'

במערכת יציבה, מספר המקטעים התפוסים (N) בממוצע גדול פי 2 ממספר החורים הפנויים (M).

הנוסחה: $N \approx 2M$

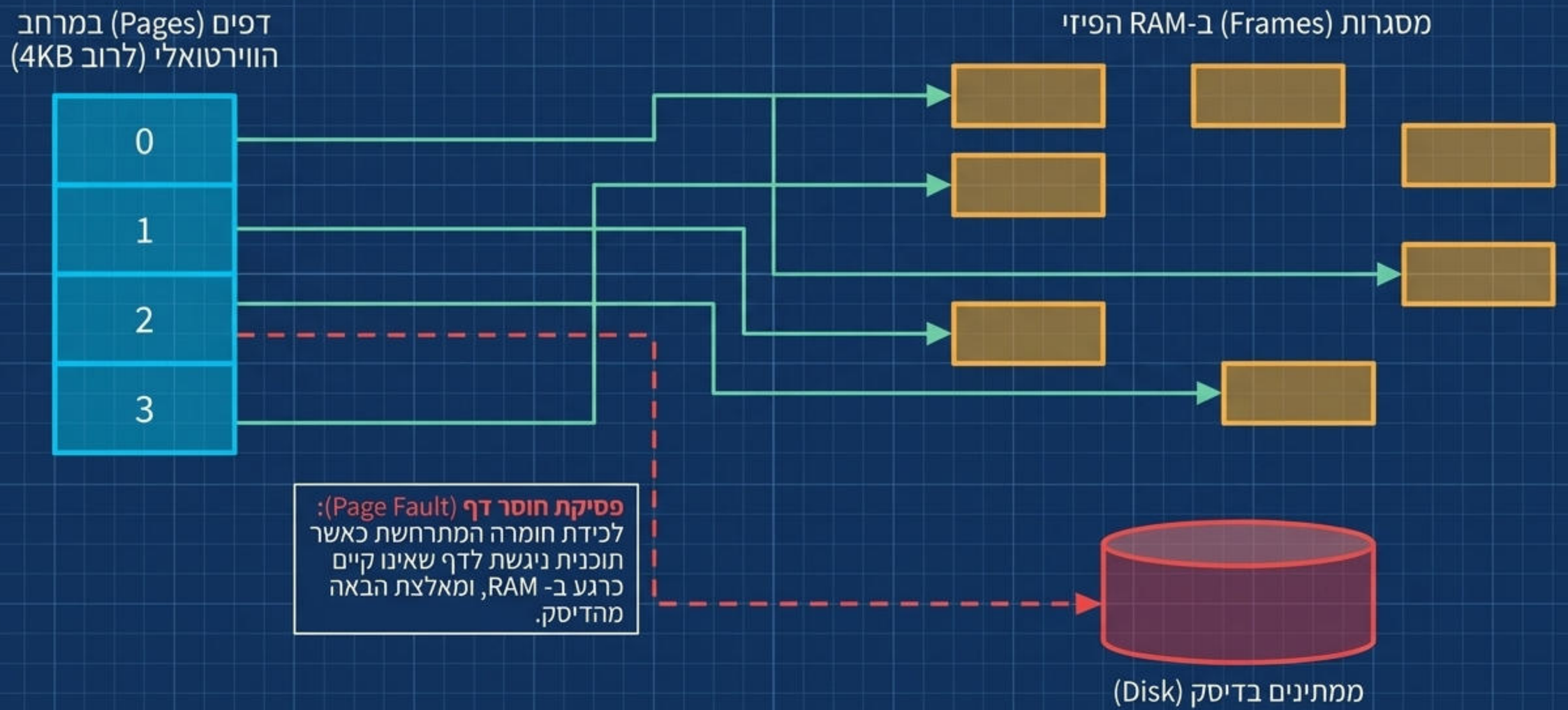
https://operating-system-5l3.pages.dev/

אלגוריתמים להקצאת זיכרון רציף

אלגוריתם	אופן פעולה	יתרונות / חסרונות	המחשה
First Fit (התאמה ראשונה)	סורק ובוחר את החור הראשון שמתאים בגודלו.	מהיר ויעיל מאוד. הגישה הנפוצה ביותר.	
Next Fit (התאמה הבאה)	כמו First Fit, אך מתחיל מאותה נקודה שבה עצר בסריקה הקודמת.	נוטה להראות ביצועים פחות טובים מ-First Fit.	
Best Fit (התאמה מיטבית)	סורק הכל ובוחר את החור הקטן ביותר שמתאים.	יוצר המון שברי זיכרון קטנטנים (פרגמנטציה חיצונית קשה).	
Worst Fit (התאמה גרועה)	סורק הכל ובוחר בכוונה את החור הגדול ביותר.	משאיר שאריות גדולות ביצועים בינוניים.	
Quick Fit (התאמה מהירה)	רשימות נפרדות לגדלים נפוצים (למשל 4KB, 8KB).	מהיר במיוחד (O(1)), אך מיזוג חורים הופך ליקר ומורכב.	
Buddy System (שיטת החברים)	הקצאה בחזקות של 2 בלבד ופיצול לחצאים.	קל למיזוג, אך יוצר פרגמנטציה פנימית כבדה.	

המהפכה: זיכרון מדומה ודפדוף (Paging)

הפתרון המוחלט לפרגמנטציה חיצונית: חיתוך הזיכרון לחתיכות קבועות. אין יותר צורך ברצף פיזי!



מבנה טבלת הדפים ומנגנון ההאצה (TLB)



Frame Number
המיקום הפיזי של הדף

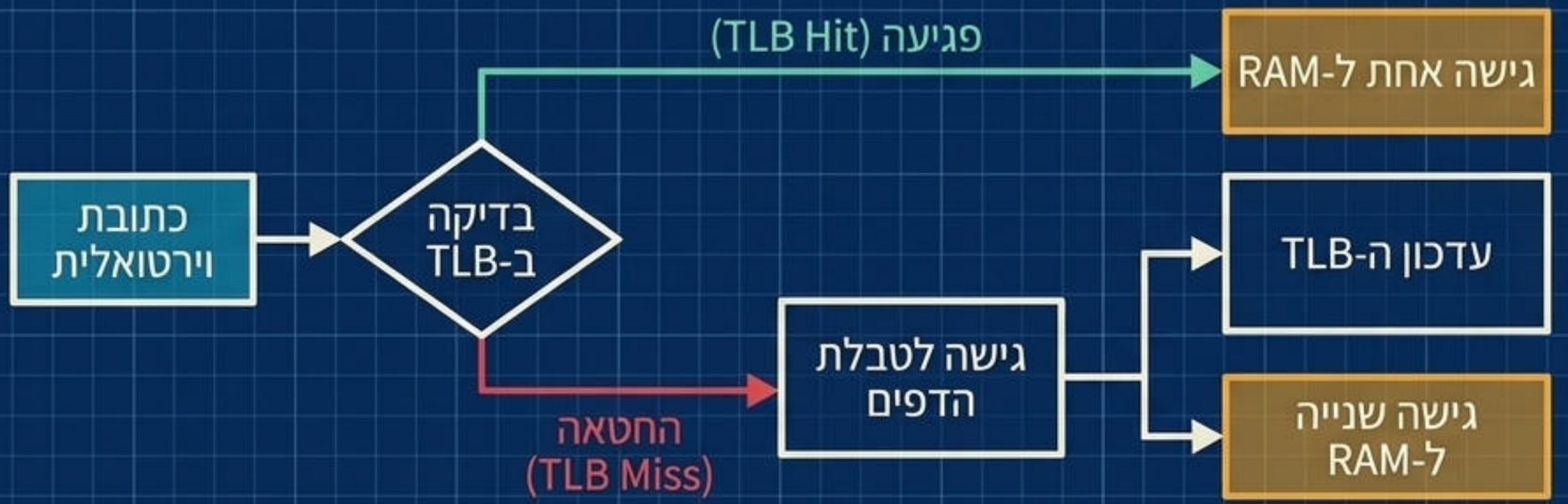
Present Bit
(1) ב-RAM
או בדיסק (0)

Protection
הרשאות גישה (R/W/X)

Reference (R)
נדלקת בגישה, קריטי להחלפה

Modified (M)
דף מלוכלך, מחייב כתיבה לדיסק

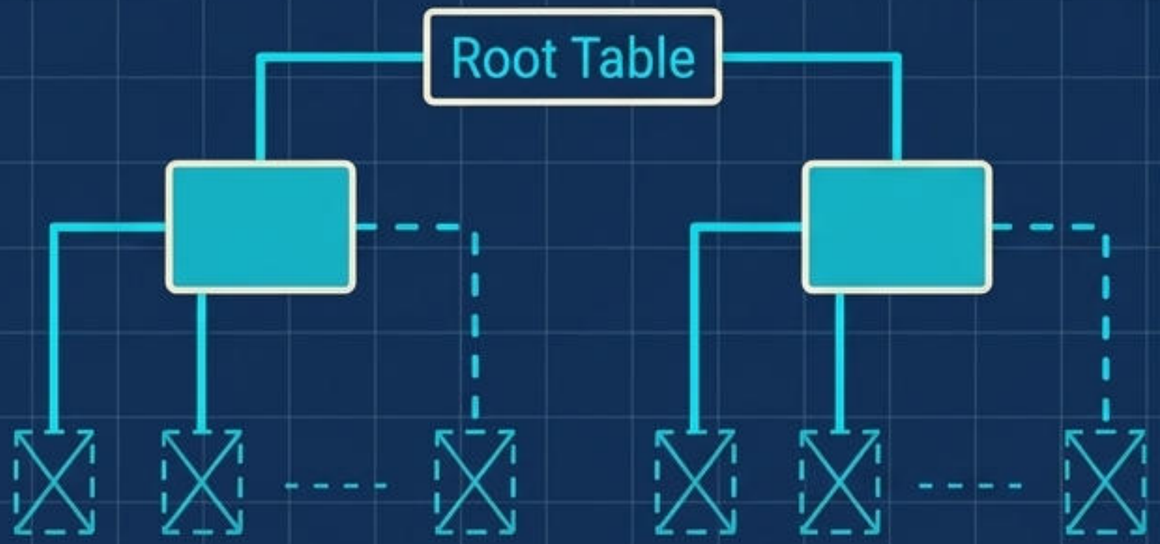
Cache Disabled
חיוני ל-Memory-Mapped I/O



מערכת ללא TLB קונסת את המעבד ב-2 גישות לזיכרון עבור כל קריאת משתנה. ה-TLB פותר זאת בעזרת עקרון המקומיות.

סקיילינג למערכות ענק ותרגום כתובות

טבלת דפים שכבתית (Multilevel)



חוסך מקום רב על ידי אי-הקצאת טבלאות לחללים וירטואלים ריקים (נפוץ ב-32-bit).

טבלה מהופכת (Inverted)

ל-64-bit, שורה אחת לכל מסגרת פיזית עם פונקציית Hash.

מעבדת מתמטיקה: תרגום כתובת

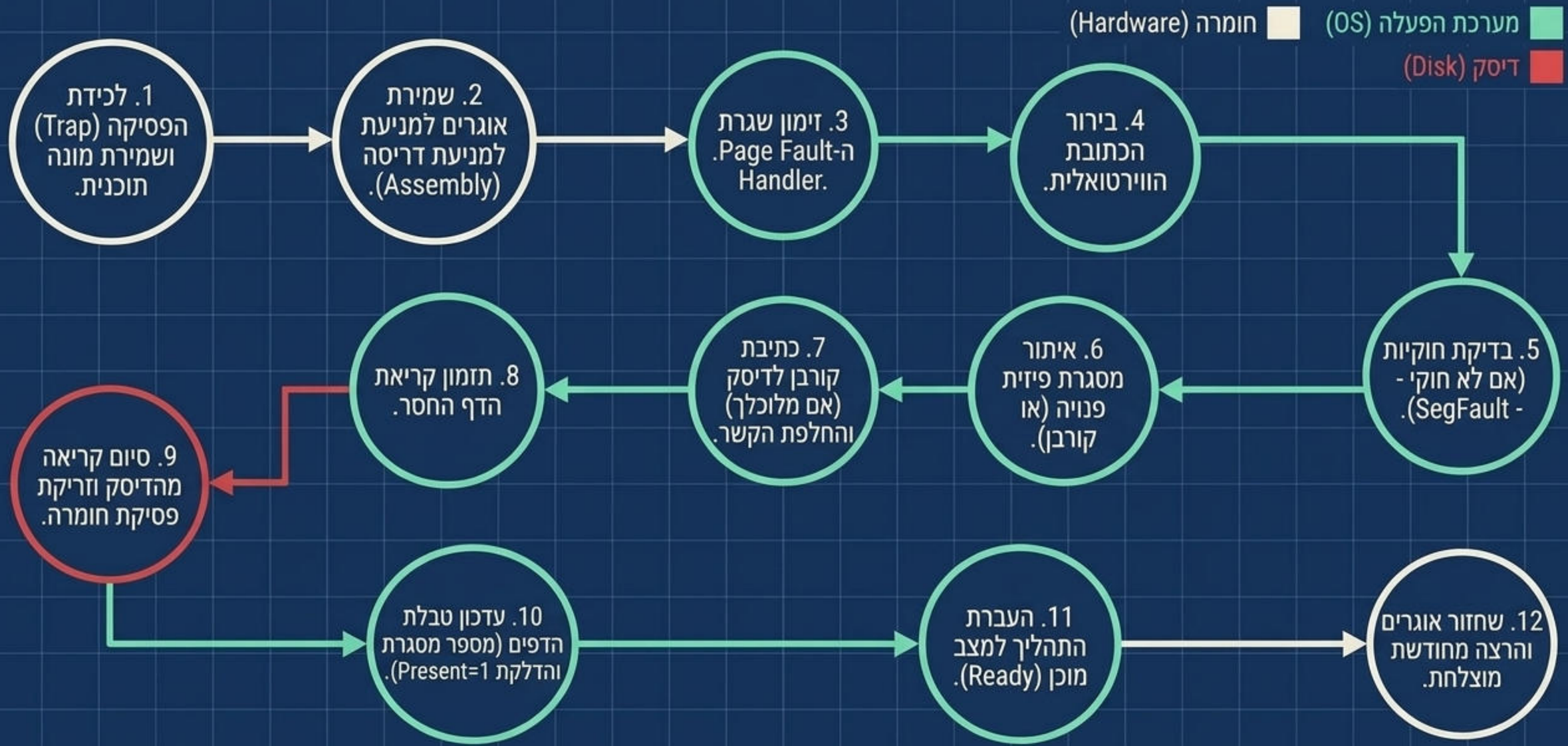
נתונים: גודל דף = 1024, דף 1 ממופה למסגרת 6.
יעד: כתובת וירטואלית 1097.

שלב 1: מציאת הדף
(דף מס' 1) 1 = 1097 / 1024

שלב 2: מציאת ההיסט
(היסט) 73 = 1097 modulo 1024

שלב 3: חישוב פיזי
Physical = (Frame * PageSize) + Offset
6217 = 73 + (6 * 1024)

האנטומיה המלאה של פסיקת חוסר דף (12 השלבים)



https://operating-system-513.pages.dev/

אסטרטגיות החלפת דפים: הבסיס והתפתחות מנגנון השעון

האופטימלי (Optimal)

פנה את הדף שיידרש בעתיד הרחוק ביותר.
משמש רק כמדד ייחוס תיאורטי (בלתי אפשרי ליישום).

NRU (Not Recently Used)

מחלק ל-4 קבוצות לפי סיביות R ו-M.
מפנה אקראית מהקבוצה הנמוכה ביותר.

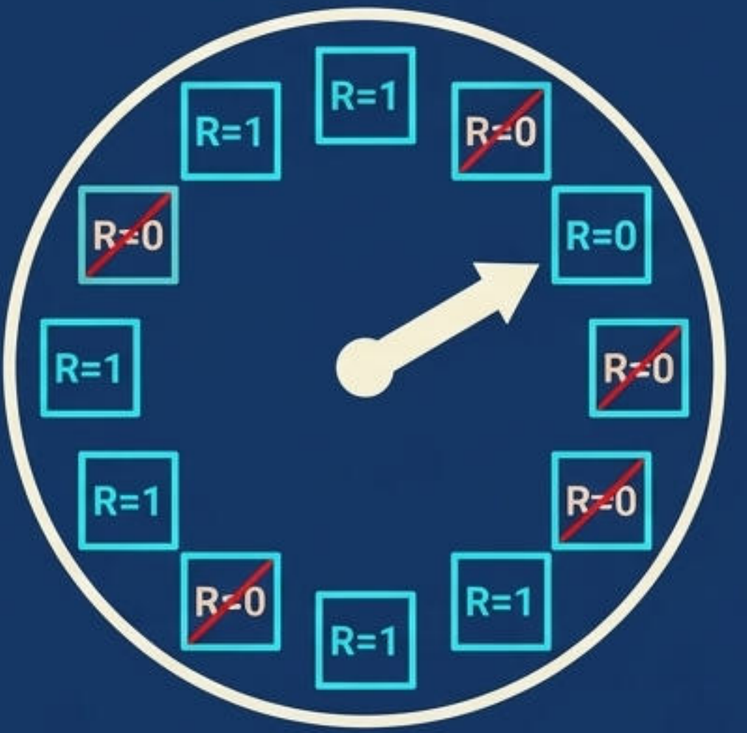
FIFO (First-In, First-Out)

מפנה את הדף הוותיק ביותר.
פגם לוגי: מפנה דף קריטי פעיל רק בגלל גילו.

הזדמנות שנייה (Second Chance)

שכלול ל-FIFO.
בודק את הדף: אם R=1, מאפס ומועבר לסוף התור.
חיסרון: מניפולציות יקרות של רשימה מקושרת.

השעון (Clock)



ייעול ארכיטקטוני של הזדמנות שנייה. הדפים במערך מעגלי
ומחוג מצביע על הבא לבדיקה.
חוסך העברת איברים בזיכרון.
אם R=1 מאפסים ומקוקדמים מחוג. אם R=0 מפנים.

https://operating-system-5l3.pages.dev/

אסטרטגיות מתקדמות: סימולציית LRU וקבוצת העבודה

LRU והתיישנות (Aging)

LRU אידיאלי: דורש חומרה יקרה לפינוי דף שלא היה בשימוש הזמן הרב ביותר.



סימולציה תוכנתית: מזיזים את המונה ימינה (מחלקים ב-2) ושותלים R ב-MSB. הקורבן הוא הערך המינימלי.

קבוצת העבודה (WS) ו-WSClock



Working Set (WS): הדפים שהתהליך ניגש אליהם כרגע.

WSClock: הסטנדרט התעשייתי. מחוג מבוסס שעון המודד את גיל ה-WS. אם דף זקן נקי מועף; אם מלוכלך מתזמנים כתיבה לדיסק והמחוג ממשיך ללא עצירת התהליך.

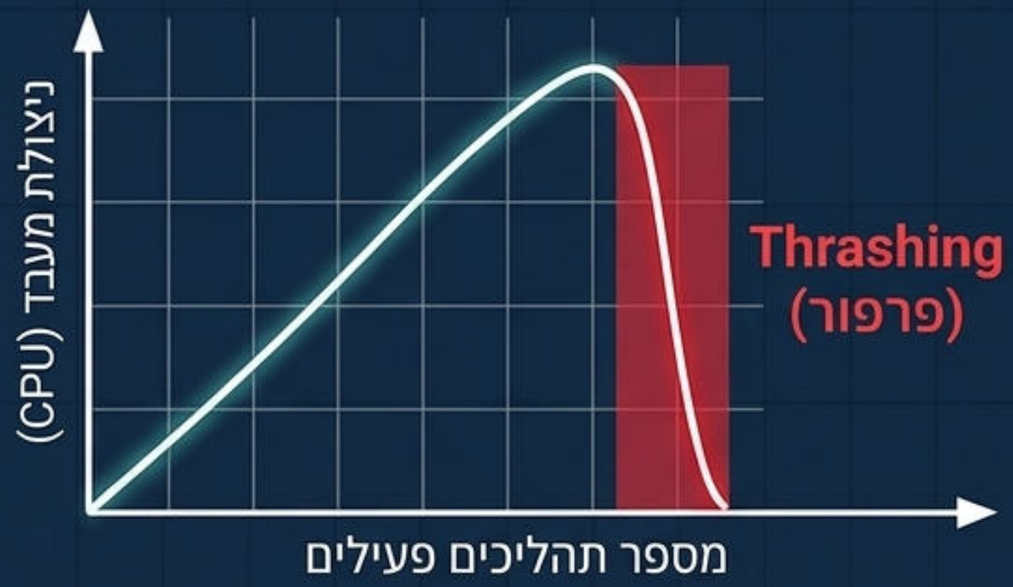
האנומליה של FIFO=LRU

בהינתן סדרת הפניות ספציפית ומורכבת ל-4 מסגרות, מתרחשת אנומליה שבה האלגוריתם הנאיבי (FIFO) והחכם (LRU) מניבים זהות מוחלטת בביצועים: 17 פסיקות דף לשניהם.

זאת בגלל חפיפת נקודות התורפה בתבנית הגישה.

https://operating-system-5l3.pages.dev/

בקרת עומס, פוליסות ומניעת קריסה (Thrashing)



כאשר סך דרישות הזיכרון חורג מה-RAM, המעבד מבלה 100% מזמנו בהחלפת דפים במקום בחישובים.
הפתרון (PFF): ה-OS מגדירה חסמים לתדירות פסיקות פסיקות. בחריגה מושהה תהליך שלם (Swap Out).

הקצאה גלובלית עדיפה על לוקאלית ליעילות המערכת.

מהו גודל הדף האידיאלי?

$$p = \sqrt{2 \cdot s \cdot e}$$

- p — גודל דף
- s — גודל תהליך
- e — גודל רשומה בטבלה

איזון עדין בין פרגמנטציה פנימית (המעודדת דפים קטנים) לגודל טבלאת/TLB (המעודדים דפים גדולים).

אדריכלות מתקדמת בסביבת ה-OS

נעילת דפים (Pinning / DMA)



חומרת I/O ניגשת ישירות ל-RAM.

ה-OS נועלת את הדף המיועד להעתקה כדי למנוע את פינויו ודריסת נתוני תוכנית אחרת על ידי ה-DMA.

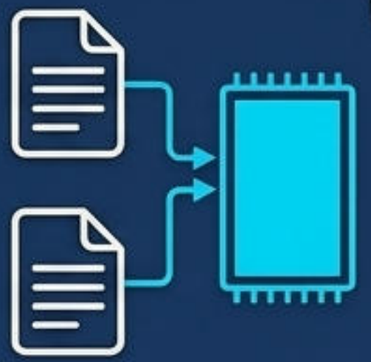
שחזור פקודות (Instruction Backup)



פסיקת דף מתרחשת באמצע פקודת מעבד.

חובה לגבות אוגרים חומרתית (Shadow Registers) בטרם כשל כדי לא להשחית נתונים בהרצה מחודשת.

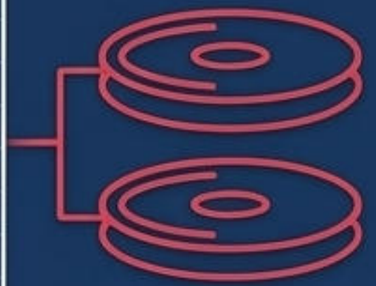
ספריות ומיפוי זיכרון (MMF)



טעינת עותק יחיד של ספרייה עם קוד PIC.

מיפוי קבצים ישירות לכתובות הווירטואליות מאיץ קריאות מהדיסק ומאפשר זיכרון משותף (IPC) מיידי בין תהליכים.

מדיניות אחסון (Backing Store)

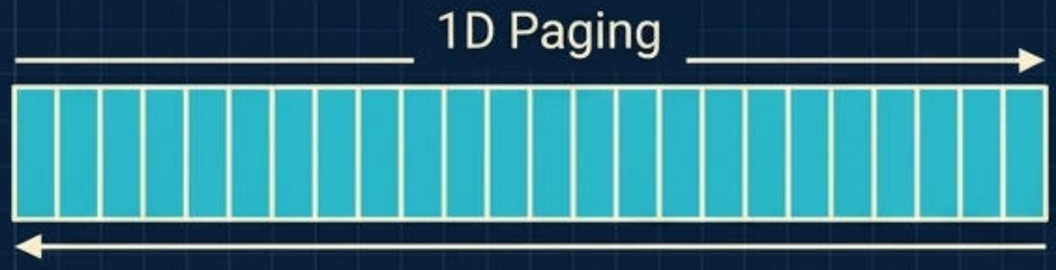


השוואה: **Swap Partition** - מחיצה מהירה ורציפה אך קשיחה (Linux).

לעומתה **Swap File** - קובץ רגיל גמיש אך אטי יותר בגלל תקורת ה-FS.

https://operating-system-5l3.pages.dev/

ארגון זיכרון לוגי: קיטוע (Segmentation) ודפדוף משולב





הבעיה בקיטוע טהור: מגן מפשוע אך מחזיר את הפרגמנטציה החיצונית.
הפתרון (x86): קיטוע משולב דפדוף שבו כל סגמנט נחתך פנימית לדפים.

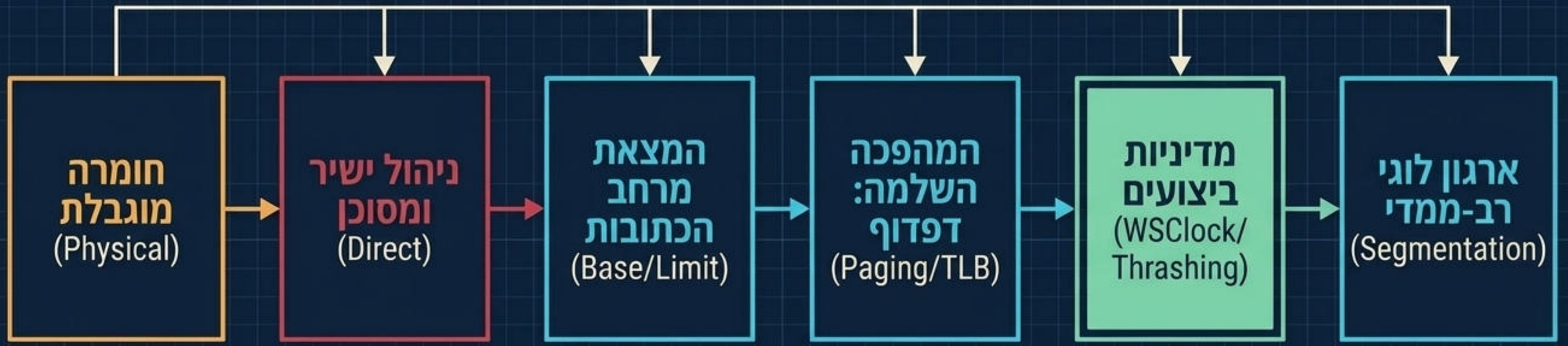
מעבדת מתמטיקה: סגמנטציה

נתונים: מרחב 4GB מפוצל ל-4 סגמנטים שווים. גודל דף 4KB.
כמה דפים בסגמנט?

1. גודל סגמנט: 1GB (2^30).
2. דפים: 2^{18} / 2^{12} = 2^{12} דפים בסגמנט (262,144).

כמה רמות דפדוף (כאשר PTE=4B) נדרשות כדי שטבלת שורש תיכנס לדף בודד?
רמה 1: מכסה את כל ה- 2^{18} דפים, ולכן דורשת 2^{8} טבלאות.
רמה 2: מכסה את 256 הרשומות של רמה 1 - נכנסת בקלות בדף יחיד המכיל 1024 רשומות.
מסקנה סופית: בדיוק 2 רמות דפדוף!

שילוב המערכות: מפת הארכיטקטורה המלאה



הפרדוקס נפתר

מערכת ההפעלה משתמשת בתרגום מתוחכם ומנגנוני חומרה מאיצים כדי לייצר אשליה של חלל עצום ופרטי לכל תהליך.
ניהול זיכרון אינו רק חלוקת שטח; הוא האמנות של יצירת יקום שלם, מבודד, ומאובטח עבור כל תוכנה על פני חומרה פיזית אחת.